

SOFTWARE REQUIREMENTS SPECIFICATION

RideAway

Smart Ride Safety Application

Version 1.1 (ML Update) | March 2026

Document Title	Software Requirements Specification
Project	RideAway – Smart Ride Safety
Version	1.1 (ML Update)
Date	March 2026
Authors	L.G.S.B. Liyanage A.J.M.R.L. Jayasundara K.R.A.R. Jayathilaka M.S.Hafsa
Standard	IEEE 830-1998 (adapted)
Platform	Flutter (Android 8.0+ / iOS 13+)
ML Framework	TensorFlow Lite (on-device inference)

Revision History

Version	Date	Author	Description
1.0	2026	RideAway Dev Team	Initial SRS
1.1	March 2026	RideAway Dev Team	Added ML model pipeline: Section 3.4 fully rewritten; NFR-ML requirements added; data model updated; Appendix C updated with ML algorithm specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for RideAway, a Flutter-based cross-platform mobile application designed to enhance personal rider safety through Machine Learning-based accident detection and automated emergency response.

Version 1.1 of this document expands the accident detection specification to reflect the decision to implement an ML-based classifier (TensorFlow Lite on-device model) rather than a fixed-threshold rule system. All relevant sections have been updated accordingly. This document is authoritative for the development team, QA engineers, and all project stakeholders.

1.2 Scope

RideAway is a mobile application targeting Android (8.0+) and iOS (13.0+). Its core innovation is an on-device Machine Learning model that classifies accelerometer and gyroscope sensor streams in real time to detect crash events, triggering an automatic emergency alert flow. The system includes six major subsystems:

- Authentication & User Management Subsystem
- Emergency Contact Management Subsystem
- Ride Session & Background Monitoring Subsystem
- ML Accident Detection Subsystem (on-device TFLite inference)
- Crash Confirmation & Emergency Notification Subsystem
- Location, Mapping & Ride History Subsystem

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
ML	Machine Learning – the use of algorithms trained on data to make predictions or decisions
TFLite	TensorFlow Lite – a lightweight ML framework for on-device inference on mobile and embedded devices
1D-CNN	One-Dimensional Convolutional Neural Network – a deep learning architecture operating on time-series data (e.g., sensor readings over time)
IMU	Inertial Measurement Unit – the combined accelerometer and gyroscope sensors in the smartphone
Sliding Window	A fixed-length segment of time-series sensor data moved forward in time by a defined step size; the unit of input to the ML model

Term	Definition
Inference	The process of running an input through a trained ML model to produce a prediction (i.e., 'crash' or 'no crash')
Confidence Score	A value between 0 and 1 output by the ML model representing the model's certainty that the input window represents a crash event
False Positive	A crash detection event triggered by a non-crash event (e.g., a speed bump classified as a crash)
False Negative	A genuine crash event that the ML model fails to classify as a crash
Confusion Matrix	An evaluation table showing the counts of true positives, false positives, true negatives, and false negatives for a classifier
TFLite Converter	A TensorFlow tool that converts a trained Keras model to the .tflite format for mobile deployment
Post-Training Quantisation	A TFLite optimisation that reduces model weight precision from float32 to int8, reducing model size and improving inference speed
GPS	Global Positioning System
SMS	Short Message Service
FCM	Firestore Cloud Messaging
Firestore	Google Cloud Firestore – NoSQL document database
G-force	A unit of acceleration equal to 9.8 m/s ² , used to characterise sensor readings
Background Service	A long-running process that continues executing while the app is not in the foreground
Confirmation Window	A 30-second on-screen timer during which the rider can dismiss a false alarm before emergency alert is sent

2. Overall Description

2.1 Product Perspective

RideAway is a standalone mobile application integrating with Firebase cloud services, device sensor APIs, and an on-device TensorFlow Lite ML model. The ML model is the system's most critical component: it processes raw IMU sensor data entirely on the device, requiring no internet connection for accident detection. This design ensures that crash detection operates reliably in remote areas with no cellular or WiFi coverage.

The ML model is trained offline in a Python/TensorFlow environment and bundled with the app as a .tflite asset. Model updates are delivered over-the-air via Firebase Remote Config without requiring a full app store update.

2.2 Product Functions (Summary)

1. User registration, authentication, and profile management
2. Emergency contact configuration and management
3. Ride session initiation and background IMU monitoring
4. On-device ML-based crash detection via TFLite inference on sensor sliding windows
5. Crash confirmation dialog with dismissal option to prevent false alarm escalation
6. Automated SMS and FCM push emergency alert dispatch
7. Live GPS location sharing and Google Maps integration
8. Ride history with ML event annotations (confidence score, outcome)

2.3 Design and Implementation Constraints

- The accident detection engine must run entirely on-device. No sensor data shall be sent to a cloud service for classification; all inference runs locally via TFLite.
- The .tflite model file must not exceed 5 MB to remain within acceptable app bundle size constraints.
- Inference latency per window must not exceed 100 ms on a minimum-spec device (Android 8, 2 GB RAM, ARM Cortex-A53).
- The ML model must be trained on a dataset that includes hard negatives: non-crash events that produce large sensor spikes (speed bumps, potholes, hard braking, curb drops).
- The background sensor service must not consume more than 5% of device battery per hour of active monitoring.
- The system must comply with Google Play Store and Apple App Store background execution policies.

2.4 Assumptions and Dependencies

- The team will assemble a labelled IMU training dataset of sufficient size (target: at least 500 crash event samples and 2,000 non-crash hard-negative samples) before model training begins.
- TFLite inference on 2-second windows at 50 Hz input is technically feasible on the minimum target device without dropping sensor frames.

-
- The `tfite_flutter` package maintains compatibility with the Flutter SDK version in use.
 - Users will carry their smartphones on their person (not mounted remotely) during rides, ensuring the device's IMU data is representative of the rider's motion.

3. Functional Requirements

3.1 Authentication & User Management

ID	Priority	Requirement
FR-AUTH-001	High	The system shall accept a full name, email, password, and password confirmation during registration.
FR-AUTH-002	High	The system shall validate email format (RFC 5322) and enforce a minimum password length of 8 characters before submission.
FR-AUTH-003	High	Upon successful registration, the system shall create a Firestore user profile under /users/{uid}.
FR-AUTH-004	High	The system shall support email/password login, returning a valid Firebase session that persists across app restarts.
FR-AUTH-005	High	The system shall support Google Sign-In. On first sign-in, a Firestore profile shall be auto-created from the Google account data.
FR-AUTH-006	High	The system shall provide a sign-out option that clears the local session and stops any active background service.
FR-AUTH-007	Medium	The system shall provide a 'Forgot Password' flow dispatching a reset email via Firebase Authentication.

3.2 Emergency Contact Management

ID	Priority	Requirement
FR-EC-001	High	The system shall provide an Emergency Contacts screen allowing the user to add contacts with name, phone number (E.164 format), and relationship label.
FR-EC-002	High	Emergency contacts shall be stored in Firestore at /users/{uid}/emergency_contacts/{contactId}.
FR-EC-003	High	The system shall allow editing and deletion of contacts, with a confirmation dialog before deletion.
FR-EC-004	Medium	The system shall support a maximum of 5 emergency contacts per user.
FR-EC-005	Medium	The system shall display a warning on the Home screen if no emergency contacts are configured.

3.3 Ride Session Management

ID	Priority	Requirement
FR-RIDE-001	High	The system shall provide a 'Start Ride' button on the Home screen that requests all required permissions before launching the background service.

ID	Priority	Requirement
FR-RIDE-002	High	Upon session start, the system shall create a Ride Session document in Firestore and begin the background sensor + ML inference loop.
FR-RIDE-003	High	The system shall display a persistent foreground notification while a Ride Session is active.
FR-RIDE-004	High	The background service shall continue operating when the app is backgrounded, the screen is locked, or another app is in the foreground.
FR-RIDE-005	High	The system shall provide a 'Stop Ride' button that terminates the session, stops the background service, and finalises the Firestore ride record.
FR-RIDE-006	Medium	The active ride screen shall display a live map with the user's GPS position, elapsed timer, and speed indicator.
FR-RIDE-007	Medium	The system shall record GPS waypoints at a minimum of one per 10 seconds for route reconstruction.

3.4 ML Accident Detection Subsystem

3.4.1 Sensor Data Acquisition

ID	Priority	Requirement
FR-ML-001	Critical	The background service shall continuously sample the device accelerometer at a minimum rate of 50 Hz (50 samples per second) during an active Ride Session.
FR-ML-002	Critical	The background service shall continuously sample the device gyroscope at a minimum rate of 50 Hz during an active Ride Session.
FR-ML-003	High	Sensor samples shall include three-axis accelerometer readings (X, Y, Z in m/s ²) and three-axis gyroscope readings (X, Y, Z in rad/s), along with a UTC timestamp for each sample.
FR-ML-004	High	The system shall maintain a circular buffer of the most recent sensor samples sufficient to fill a 2-second sliding window (100 samples per axis at 50 Hz).
FR-ML-005	Medium	The system shall implement a motion-context check: if the resultant accelerometer magnitude is below a low-motion threshold (configurable, default: 0.5 G above gravity) for 10 consecutive seconds, sensor sampling rate may be reduced to 10 Hz to conserve battery.

3.4.2 Feature Engineering

ID	Priority	Requirement
FR-ML-010	High	The system shall segment the sensor stream into overlapping sliding windows of 2 seconds duration (100 samples per axis) with a 50% overlap (advance of 1 second between windows).

ID	Priority	Requirement
FR-ML-011	High	For each window, the system shall compute the following per-axis statistical features: mean, standard deviation, minimum, maximum, and root mean square (RMS) for both accelerometer and gyroscope axes.
FR-ML-012	High	For each window, the system shall compute the resultant vector magnitude: $\sqrt{x^2 + y^2 + z^2}$ for accelerometer and gyroscope independently, and include magnitude mean, max, and standard deviation as features.
FR-ML-013	High	For each window, the system shall compute peak G-force (maximum resultant accelerometer magnitude in the window) and the rate of change of acceleration (jerk: first derivative of resultant magnitude).
FR-ML-014	Medium	For each window, the system shall compute signal energy in three frequency bands (low: 0–5 Hz, mid: 5–15 Hz, high: 15–25 Hz) for the resultant accelerometer magnitude via Fast Fourier Transform, as supplementary features for the ML model.
FR-ML-015	Medium	The assembled feature vector (target dimensionality: 30–50 features) shall be normalised using the mean and standard deviation computed on the training dataset, applied at inference time via stored normalisation parameters bundled with the model.

3.4.3 ML Model Architecture & Training (Offline Pipeline)

ID	Priority	Requirement
FR-ML-020	High	The ML model shall be a binary classifier with two output classes: class 0 = 'normal riding event' and class 1 = 'crash event'.
FR-ML-021	High	The primary model architecture shall be a 1D Convolutional Neural Network (1D-CNN) with input shape [window_size, num_axes] operating on the raw normalised sensor window. An alternative feature-based Random Forest or Gradient Boosting classifier shall be trained as a baseline for comparison.
FR-ML-022	High	The training dataset shall contain labelled examples of: (a) crash events — simulated and controlled impact scenarios across multiple device placements; (b) hard-negative non-crash events — speed bumps, potholes, hard braking, kerb drops, and phone drops.
FR-ML-023	High	The training dataset shall be split into train (70%), validation (15%), and test (15%) sets with stratified sampling to preserve class balance.
FR-ML-024	High	The model shall be evaluated on the held-out test set using: accuracy, precision, recall, F1-score, and the confusion matrix. The target performance on the test set is: recall (crash class) ≥ 0.95 and false positive rate ≤ 0.02 .
FR-ML-025	High	The trained Keras model shall be converted to TensorFlow Lite (.tflite) format using the TFLiteConverter. Post-training integer quantisation (int8) shall be applied to reduce model size and improve inference speed on ARM CPUs.
FR-ML-026	Medium	The final .tflite model file shall not exceed 5 MB in size.

ID	Priority	Requirement
FR-ML-027	Medium	All training data, model training scripts, evaluation results, and conversion scripts shall be version-controlled and documented in the project repository under /ml/.

3.4.4 On-Device Inference (Flutter / TFLite)

ID	Priority	Requirement
FR-ML-030	Critical	The .tflite model file shall be bundled as an asset within the Flutter application and loaded by the tflite_flutter package on app startup.
FR-ML-031	Critical	The background service shall run the ML inference pipeline on each new sliding window, producing a crash probability score (float, 0.0–1.0).
FR-ML-032	Critical	A crash event shall be flagged when the model output exceeds a configurable confidence threshold (default: 0.85) for two consecutive overlapping windows. The consecutive-window requirement provides temporal consistency filtering to further suppress false positives.
FR-ML-033	High	The confidence threshold shall be user-configurable within the range 0.70–0.95 via the Settings screen.
FR-ML-034	High	The ML inference time per window shall not exceed 100 ms on the minimum target device specification (Android 8, ARM Cortex-A53, 2 GB RAM).
FR-ML-035	High	Upon a flagged crash event, the system shall log the event to Firestore immediately with: timestamp, GPS coordinates, peak G-force, peak gyroscope magnitude, and the ML confidence score of the triggering window.
FR-ML-036	Medium	The system shall provide a 'Run Detection Test' mode in Settings that allows the user to simulate a crash event (by shaking the device in a prescribed pattern) to verify that the ML engine is active and the confirmation dialog appears, without sending alerts to emergency contacts.
FR-ML-037	Low	The system shall support over-the-air ML model updates via Firebase Remote Config. A new .tflite file URL and version identifier may be pushed remotely; the app shall download and validate the updated model (checksum verification) and switch to it on the next app restart.

3.5 Crash Confirmation & Emergency Notification

3.5.1 Confirmation Window

ID	Priority	Requirement
FR-CD-010	Critical	Upon ML crash event flagging, the system shall bring the app to the foreground and display a full-screen Confirmation Alert Dialog.

ID	Priority	Requirement
FR-CD-011	Critical	The dialog shall display a countdown timer (default: 30 seconds, user-configurable 15–60 seconds) with large, high-contrast typography readable in outdoor daylight.
FR-CD-012	Critical	The dialog shall emit an audible alarm and vibration even if the device is in silent mode.
FR-CD-013	Critical	A clearly labelled 'I AM OK – Cancel Alert' button shall dismiss the dialog, cancel the emergency flow, and update the Firestore event record outcome to 'dismissed'.
FR-CD-014	Critical	If the countdown reaches zero without user interaction, the system shall automatically proceed to emergency notification dispatch.
FR-CD-015	Medium	The dialog shall display the ML confidence score of the triggering event in a technical details section (expandable, for power users).

3.5.2 SMS & Push Alerts

ID	Priority	Requirement
FR-NOTIF-001	Critical	Upon confirmation window timeout, the system shall dispatch an SMS to every registered emergency contact containing: rider name, crash alert text, date/time, and a Google Maps URL of the last-known GPS coordinates.
FR-NOTIF-002	High	SMS dispatch shall use the device's native telephony API and shall not require internet connectivity.
FR-NOTIF-003	High	If SMS dispatch fails, the system shall retry up to 3 times at 60-second intervals.
FR-NOTIF-004	Medium	If the emergency contact has RideAway installed, the system shall also dispatch an FCM push notification with a deep link to the rider's map location.
FR-NOTIF-005	Medium	The system shall log SMS delivery status (sent/failed) and final alert outcome per contact in the Firestore event record.

3.6 Location & Mapping

ID	Priority	Requirement
FR-LOC-001	High	The system shall request background location permission before the first ride session.
FR-LOC-002	High	During an active session, the system shall stream GPS coordinates at minimum once per 5 seconds when the device is in motion.
FR-LOC-003	High	The active ride screen shall display the user's position on an embedded Google Map.
FR-LOC-004	High	The system shall cache the last known GPS coordinates locally for use in emergency alerts if live GPS is temporarily unavailable.

ID	Priority	Requirement
FR-LOC-005	Medium	Completed ride routes shall be displayed as a polyline on a static map in the Ride History detail screen.

3.7 Ride History

ID	Priority	Requirement
FR-HIST-001	Medium	The system shall store a completed Ride Session in Firestore containing: start/end timestamps, total distance, GPS waypoints array, and a list of ML detection events.
FR-HIST-002	Medium	Each ML detection event record in the ride history shall include: timestamp, GPS coordinates, peak G-force, peak gyroscope magnitude, ML confidence score, and outcome (alert_sent / dismissed / auto_dismissed).
FR-HIST-003	Medium	The Ride History screen shall display a chronological list of past rides with date, duration, distance, and incident count.
FR-HIST-004	Medium	A ride detail view shall show the route map, statistics, and an annotated timeline of ML-flagged events during the session.

4. Non-Functional Requirements

4.1 ML Model Performance

ID	Category	Requirement
NFR-ML-001	Accuracy	The ML model shall achieve a recall (sensitivity) of at least 0.95 on the held-out crash class test set, meaning no more than 5% of genuine crash events are missed.
NFR-ML-002	Precision	The ML model shall achieve a false positive rate of no more than 2% on the held-out non-crash hard-negative test set.
NFR-ML-003	Latency	On-device TFLite inference per 2-second sliding window shall complete in under 100 ms on the minimum target device (Android 8, ARM Cortex-A53, 2 GB RAM).
NFR-ML-004	Model Size	The bundled .tflite model file shall not exceed 5 MB after post-training int8 quantisation.
NFR-ML-005	Generalisation	The model shall be evaluated on data from at least 3 different device placements (trouser pocket, jacket pocket, handlebar mount) to demonstrate placement-invariant performance.
NFR-ML-006	Retrainability	The ML training pipeline (data preprocessing, model training, TFLite conversion) shall be fully documented and reproducible from the version-controlled /ml/ directory using a requirements.txt-pinned Python environment.
NFR-ML-007	OTA Updates	The Firebase Remote Config OTA model update mechanism shall include checksum (SHA-256) verification of the downloaded .tflite file before activating it. A failed checksum shall fall back to the bundled model.

4.2 Performance

ID	Category	Requirement
NFR-PERF-001	Response Time	The app shall launch to the Home screen within 3 seconds on a mid-range device.
NFR-PERF-002	Detection Speed	The full pipeline — sensor window complete → feature extraction → TFLite inference → crash flag — shall complete within 200 ms.
NFR-PERF-003	Alert Speed	Emergency SMS dispatch shall begin within 2 seconds of confirmation window timeout.
NFR-PERF-004	Battery	The combined background sensor sampling + ML inference loop shall consume no more than 5% of device battery per hour on a 4,000 mAh device.
NFR-PERF-005	Memory	The app shall consume no more than 150 MB RAM during an active ride session with live map and ML inference running concurrently.

4.3 Reliability

ID	Category	Requirement
NFR-REL-001	Availability	The ML inference pipeline shall be operational 100% of the time during an active Ride Session, independent of internet connectivity.
NFR-REL-002	Crash Detection	The implemented system (ML model + consecutive-window confirmation) shall demonstrate a false positive rate below 2% on a standardised test scenario suite run on physical devices.
NFR-REL-003	Service Recovery	If the background service is terminated by the OS, it shall automatically restart within 30 seconds.
NFR-REL-004	Alert Recovery	In the event of a network failure during emergency notification, the system shall queue the alert and retry upon connectivity restoration for up to 24 hours.

4.4 Security

ID	Category	Requirement
NFR-SEC-001	Data Protection	All Firestore data including ML event logs (GPS coordinates, confidence scores) shall be protected by security rules enforcing authenticated-user-only access.
NFR-SEC-002	Transport	All Firebase communication shall use TLS 1.2 or higher.
NFR-SEC-003	ML Model	The bundled .tflite model file shall not expose training data. No personally identifiable information is embedded in the model weights.
NFR-SEC-004	OTA Integrity	Downloaded .tflite model updates shall be validated against a SHA-256 checksum stored in Firebase Remote Config before activation.

4.5 Usability

ID	Category	Requirement
NFR-USE-001	Setup Time	A first-time user shall be able to complete registration, add one emergency contact, and start their first ride session within 5 minutes.
NFR-USE-002	Alert Dialog	The Confirmation Alert Dialog shall use a minimum font size of 28 sp and a contrast ratio of at least 4.5:1 to be readable in outdoor daylight.
NFR-USE-003	Accessibility	All interactive elements shall have a minimum tap target of 44x44 dp. TalkBack and VoiceOver screen reader labels shall be applied to all UI elements.

ID	Category	Requirement
NFR-USE-004	ML Transparency	The Settings screen shall display the currently loaded ML model version number and the active confidence threshold, so technically aware users can verify the system state.

5. External Interface Requirements

5.1 User Interface Requirements

Screen	Description & Key UI Elements
Home Screen	Active session status. 'Start Ride' / 'Stop Ride' CTA. Warning if no contacts. Bottom nav.
Active Ride Screen	Fullscreen Google Map, live GPS marker, session timer, speed indicator, 'Stop Ride' button, manual SOS button.
Confirmation Alert Dialog	Full-screen overlay. Large countdown timer. Alarm indicator. 'I AM OK' dismiss button. ML confidence score (expandable).
Emergency Contacts	Contact list with name/phone/relationship. FAB to add. Swipe-to-delete. Edit on tap.
Ride History Screen	Chronological ride list with date, duration, distance, incident badge count.
Ride Detail Screen	Static route map. Summary stats. ML event timeline with confidence scores and outcomes.
Settings Screen	ML confidence threshold slider (0.70–0.95). Confirmation window duration (15–60s). Alert preferences. Model version display. 'Run Detection Test' button. Notification toggles.
Profile Screen	User info, edit profile, sign out, account deletion.

5.2 Hardware Interface Requirements

- **3-axis linear accelerometer required at minimum 50 Hz. Used as primary input to the ML model.:** Accelerometer
- **3-axis gyroscope required at minimum 50 Hz. Used as secondary input to the ML model for rotational event characterisation.:** Gyroscope
- **TFLite inference runs on the device CPU. Minimum: ARM Cortex-A53 (Android 8-era). The quantised int8 model is optimised for ARM NEON SIMD instructions.:** CPU (ARM)
- **Required for live location and emergency alert coordinate embedding. Minimum accuracy: 20 m.:** GPS / GNSS
- **Required for SMS emergency alert dispatch via native telephony API.:** Cellular Modem
- **Required for Confirmation Alert Dialog alarm sound and vibration.:** Speaker/Haptics

5.3 Software Interface Requirements

- **Bundled via tflite_flutter. The interpreter loads the .tflite asset, manages input/output tensor buffers, and executes the model graph.:** TensorFlow Lite Runtime
- **Provides SensorEvent streams for accelerometer and gyroscope at 50 Hz via platform channel to the native Android/iOS sensor APIs.:** sensors_plus

-
- **Used to push ML model version identifiers, download URLs, and confidence threshold overrides to the live app without an app store update.:** `firebase_remote_config`
 - **User session management.:** Firebase Authentication
 - **Stores user profiles, emergency contacts, ride sessions, and ML event logs.:** Cloud Firestore
 - **Embedded map for live tracking and ride history route display.:** Google Maps Flutter SDK
 - **Manages the foreground service lifecycle that hosts the sensor sampling and ML inference loop.:** `flutter_background_service`

6. Data Model (Firestore Schema)

RideAway uses Firestore collections. All documents include automatic createdAt / updatedAt timestamps. The ML subsystem adds fields to the rides and incidents structures to capture detection events and model metadata.

6.1 /users/{uid}

Field	Type	Constraints	Description
_id / uid	String	Auto	Firestore doc ID
displayName	String	Required	Full name
email	String	Required, unique	Login email
phoneNumber	String	Optional	User's own phone (E.164)
photoURL	String?	Optional	Profile photo URL
createdAt	Timestamp	Auto	Account creation timestamp
settings	Map	See 6.4	Embedded settings including ML thresholds

6.2 /users/{uid}/emergency_contacts/{contactId}

Field	Type	Constraints	Description
contactId	String	Auto	Firestore doc ID
name	String	Required	Contact full name
phoneNumber	String	Required, E.164	SMS target number
relationship	String?	Optional	Relationship label

6.3 /users/{uid}/rides/{rideId}

Field	Type	Constraints	Description
rideId	String	Auto	Firestore doc ID
startTime	Timestamp	Required	Session start UTC
endTime	Timestamp?	Null if active	Session end UTC
distanceKm	Number	Required	Total km covered
waypoints	Array<GeoPoint>	Required	GPS waypoint array
status	String	active completed interrupted	Session status

Field	Type	Constraints	Description
mlModelVersion	String	Required	Version ID of the TFLite model active during this session
incidents	Array<Map>	See 6.3.1	ML detection events during this session

6.3.1 Incident Sub-Document (within rides.incidents)

Field	Type	Constraints	Description
detectedAt	Timestamp	Required	UTC timestamp of ML detection flag
location	GeoPoint	Required	GPS at detection time
mlConfidenceScore	Number	0.0–1.0, Required	ML model output score for the triggering window
mlWindowFeatures	Map?	Optional	Extracted feature summary (peak G-force, peak rotation, RMS) for post-hoc analysis
peakAcceleration	Number	Required	Max resultant G-force in triggering window
peakRotation	Number	Required	Max gyroscope magnitude (rad/s)
outcome	String	alert_sent dismissed auto_dismissed	Final event outcome
alertsSentTo	Array<String>	Required if alert_sent	Phone numbers of contacts who received SMS
smsDeliveryStatus	Map<String,String>	Optional	Map of phoneNumber → 'sent' 'failed'

6.4 Settings Map (embedded in /users/{uid})

Field	Type	Default	Description
mlConfidenceThreshold	Number	0.85	ML crash classification confidence threshold (0.70–0.95)
mlModelVersion	String	bundled	Active TFLite model version; 'bundled' = app-packaged model

Field	Type	Default	Description
consecutiveWindowsRequired	Integer	2	Number of consecutive windows above threshold required to flag crash
confirmationWindowSecs	Integer	30	Countdown seconds before auto-alert dispatch (15–60)
smsAlertsEnabled	Boolean	true	Whether to send SMS on confirmed crash
pushAlertsEnabled	Boolean	true	Whether to send FCM push notifications
soundEnabled	Boolean	true	Alarm sound in confirmation dialog
vibrationEnabled	Boolean	true	Vibration in confirmation dialog

7. Use Cases

UC-001: New User Registration

Use Case ID	UC-001
Title	New User Registration via Email
Actor	Unauthenticated User
Preconditions	User has no existing account. Internet available.
Main Flow	1. User opens app → Splash → Onboarding → Registration. 2. User enters name, email, password, confirm password. 3. User submits. System validates inputs. 4. System creates Firebase Auth user and Firestore profile. 5. User navigated to Home screen.
Alternate Flows	3a. Email already exists: 409 error shown. 3b. Passwords don't match: inline error shown.
Postconditions	User authenticated. Firestore profile created. User on Home screen.

UC-002: ML Crash Detection and Emergency Alert

Use Case ID	UC-002
Title	ML Model Detects Crash – Emergency Alert Dispatched
Actor	System (automated) / Rider (confirmation window)
Preconditions	Active Ride Session running. At least one emergency contact registered. ML model loaded.
Main Flow	1. Background service samples accelerometer + gyroscope at 50 Hz. 2. Each second, a new 2-second sliding window is assembled. 3. Feature vector extracted from window (per-axis stats, magnitude features, jerk). 4. Feature vector normalised using bundled normalisation parameters. 5. TFLite model runs inference; outputs crash confidence score. 6. Score exceeds configured threshold (e.g., 0.85). 7. Next consecutive window also exceeds threshold. 8. Crash event flagged. Incident document written to Firestore with confidence score and GPS. 9. App brought to foreground. Confirmation Alert Dialog displayed with 30-second countdown and alarm. 10. Countdown reaches zero with no user interaction. 11. System captures current GPS. Sends SMS to all emergency contacts. 12. FCM push sent if applicable. 13. Incident outcome updated to 'alert_sent'. Dialog closes. Ride session continues.
Alternate Flows	7a. Second window score is below threshold: consecutive window count resets. No crash flagged. 9a. Rider taps 'I AM OK': Dialog dismissed. Incident outcome set to 'dismissed'. No SMS sent. 11a. GPS unavailable: Last cached coordinates used. If none, SMS states 'GPS unavailable'. 11b. SMS fails: Retry queued. Up to 3 retries at 60-second intervals.
Postconditions	Emergency contacts have received SMS. ML incident logged in Firestore with full feature data. Ride continues.

UC-003: User Adjusts ML Confidence Threshold

Use Case ID	UC-003
Title	User Configures ML Detection Sensitivity
Actor	Authenticated Rider
Preconditions	User is authenticated and familiar with receiving false positives or concerned about missed detections.
Main Flow	1. User navigates to Settings screen. 2. User views current ML Confidence Threshold value (default: 0.85) and current model version. 3. User moves slider to a higher value (e.g., 0.92) to reduce false positives. 4. System saves updated threshold to Firestore /users/{uid}/settings.mlConfidenceThreshold. 5. On next ride session, the background service applies the new threshold in the inference pipeline. 6. User taps 'Run Detection Test'. Confirmation dialog appears on shake input without sending SMS.
Postconditions	New threshold persisted. Applied to all subsequent ride sessions. Detection test confirms system is operational.

8. System Constraints & Compliance

8.1 ML-Specific Constraints

- The ML model must run entirely on-device. No sensor data shall be transmitted to any external service for the purpose of crash classification.
- The training dataset must not contain personally identifiable sensor data from real accident victims. All training data is either simulated, sourced from public research datasets (SisFall, MobiFall), or recorded with informed consent from team members.
- The ML model's training data, evaluation metrics (confusion matrix, F1, recall), and conversion scripts must be version-controlled and reproducible.
- The app shall clearly disclose in its Privacy Policy that anonymised crash event sensor data (not raw streams) may be collected with user consent to improve the ML model in future versions.

8.2 Platform Constraints

- Android: Foreground Service with `FOREGROUND_SERVICE_TYPE_LOCATION`. Battery optimisation exemption via `IGNORE_BATTERY_OPTIMIZATIONS`. `SERVICE_STICKY` restart behaviour.
- iOS: Background Modes — 'Location updates' and 'Background fetch' declared in `Info.plist`. Core Motion framework access via `sensors_plus`.
- App Stores: Background activity, sensor usage, and location access must comply with Google Play and Apple App Store review policies including required permission usage descriptions.

8.3 Privacy & Regulatory Compliance

- GDPR / PDPA: Privacy Policy must disclose sensor data collection, ML processing, and emergency contact data storage. Right to erasure (account deletion) must remove all Firestore records including ride history and ML event logs.
- Data Minimisation: The ML inference runs entirely on-device. Raw sensor streams are never transmitted to the cloud. Only extracted event summaries (peak G-force, confidence score, GPS) are logged to Firestore.

9. Appendices

Appendix A: Requirements Traceability Matrix

Req ID	Requirement Summary	Test Case ID	Status
FR-AUTH-001	Registration accepts required fields	TC-AUTH-001	Implemented
FR-AUTH-005	Google Sign-In creates Firestore profile	TC-AUTH-005	Implemented
FR-EC-001	Emergency contacts CRUD	TC-EC-001	In Progress
FR-RIDE-001	Start Ride launches background service	TC-RIDE-001	Planned
FR-ML-001	50 Hz accelerometer sampling	TC-ML-001	Planned
FR-ML-010	2-second sliding window segmentation	TC-ML-010	Planned
FR-ML-011	Per-axis statistical feature extraction	TC-ML-011	Planned
FR-ML-020	Binary crash classifier output	TC-ML-020	Planned
FR-ML-024	Model recall ≥ 0.95 on test set	TC-ML-024	Planned
FR-ML-030	TFLite model bundled and loaded at startup	TC-ML-030	Planned
FR-ML-031	Inference runs per sliding window	TC-ML-031	Planned
FR-ML-032	Consecutive-window crash confirmation logic	TC-ML-032	Planned
FR-ML-037	OTA model update with checksum validation	TC-ML-037	Planned
FR-CD-010	Confirmation dialog on ML flag	TC-CD-010	Planned
FR-NOTIF-001	SMS dispatched to all contacts on timeout	TC-NOTIF-001	Planned
NFR-ML-001	Recall ≥ 0.95 on crash test set	TC-NFR-ML-001	Planned
NFR-ML-002	False positive rate ≤ 0.02	TC-NFR-ML-002	Planned
NFR-ML-003	Inference < 100 ms on min-spec device	TC-NFR-ML-003	Planned
NFR-SEC-001	Firestore rules enforce auth-only access	TC-SEC-001	Implemented

Appendix B: ML Model Algorithm Specification

The following describes the crash detection ML pipeline from raw sensor input to classification output.

9. Sensor Input: At each 1-second step, a new sliding window W of shape $[100, 6]$ is assembled (100 samples $\times$ 6 axes: acc_x , acc_y , acc_z , $gyro_x$, $gyro_y$, $gyro_z$) sampled at 50 Hz.
10. Normalisation: Each axis value is standardised: $v_norm = (v - \mu_axis) / \sigma_axis$, where μ and σ are computed on the training dataset and stored as constants bundled with the .tflite model.
11. Feature Extraction (for feature-based path): Compute per-axis mean, std, min, max, RMS; resultant vector magnitudes; peak G-force; jerk; FFT energy bands. Assemble into feature vector F of dimension ~ 40 .
12. Inference: Feed normalised window W (1D-CNN path) or feature vector F (RF path) to the TFLite interpreter. Interpreter returns crash probability p in $[0, 1]$.
13. Consecutive-Window Filter: Maintain a counter C . If $p > \text{threshold}$: $C++$. If $C \geq 2$: flag crash event. If $p \leq \text{threshold}$: $C = 0$ (reset).
14. Crash Event: Write incident document to Firestore with timestamp, GPS, p value, peak G-force, peak rotation. Trigger Confirmation Alert Dialog.

Target performance on held-out test set: Recall (crash class) ≥ 0.95 . FPR (non-crash class) ≤ 0.02 . Inference latency per window ≤ 100 ms on ARM Cortex-A53.

Appendix C: ML Training Data Sources

- **A publicly available dataset of simulated falls and activities of daily living collected with IMU sensors. Provides negative (non-crash) activity samples and fall events as proxy crash data.:** SisFall Dataset
- **Smartphone-based fall detection dataset. Provides labelled fall events from multiple body positions including trouser pocket and shirt pocket.:** MobiFall Dataset
- **Controlled crash simulations: bicycle and scooter drops at various speeds, device in trouser pocket and handlebar mount. Non-crash hard negatives: speed bumps at 10/20/30 km/h, pothole impacts, emergency braking, device drops onto ground.:** Team-Collected Data
- **Time warping (stretching/compressing the time axis by $\pm 10\%$), magnitude jitter (adding Gaussian noise, $\sigma = 0.05$ G), and axis rotation augmentation to increase dataset diversity and model robustness to device orientation variation.:** Data Augmentation